Hearing the Platonic Solids

Graph Sonification

Darshani P. Gole

Graph **sonification** is the process of converting mathematical properties of a graph into sound. Instead of representing a graph visually, structural information is mapped to acoustic parameters such as **pitch**, **loudness**, **rhythm**, or **timbre**. This enables graph-theoretic features to be explored through auditory perception.

Sonification is particularly useful for comparing graphs, identifying patterns in their spectra, and providing an alternative representation of complex mathematical structures.

In spectral graph theory, the spectrum of a graph contains valuable information about its connectivity and symmetry. Since eigenvalues are numerical quantities, they naturally lend themselves to mappings into musical frequencies. Likewise, eigenspaces, whose dimensions are determined by eigenvalue multiplicities, can be represented through richer harmonic structures.

Mathematical Background

Let $ G=(V,E)$ be a graph with adjacency matrix A. The **degree matrix** is

$$D = \mathrm{diag}(d_1, d_2, \dots, d_n),$$

where

$$d_i = \sum_j A_{ij}.$$

The **graph Laplacian** is defined as

$$L = D - A.$$

Since the Laplacian is a real symmetric matrix, all of its eigenvalues are real and admit an orthogonal set of eigenvectors. The Laplacian spectrum consists of the eigenvalues

$$0 = \lambda_1 \leq \lambda_2 \leq \cdots \leq \lambda_n.$$

Eigenvalues may occur with multiplicity greater than one. The multiplicity of an eigenvalue equals the dimension of its corresponding eigenspace.

The eigenvalues were computed numerically using Python. Since the Laplacian is symmetric, the numerical routines return all eigenvalues together with an orthonormal basis of eigenvectors.

Mapping Eigenvalues to Frequencies

Each eigenvalue was converted into an audible frequency using the exponential mapping

$$f = f_0 \, 2^{\lambda/\lambda_{\max}},$$

where

- f_0 is the base frequency (220 Hz in this implementation), and
- $\lambda_{\max}$ is the largest Laplacian eigenvalue.

The exponential mapping follows the logarithmic nature of musical pitch. Equal frequency ratios correspond to equal musical intervals, making this mapping perceptually more meaningful than a linear relationship.

Eigenvalue Sonification

The first sonification method represents each distinct eigenvalue by a musical tone.

For every spectral block (λ, m), the corresponding frequency is computed using the previous mapping. If an eigenvalue has multiplicity greater than one, multiple nearly identical tones are generated with slight frequency offsets,

$$f_k = f\left(1 + 0.01\left(k - \frac{m-1}{2}\right)\right),$$

for

$$k = 0, 1, \ldots, m-1.$$

Each tone is represented by the sinusoidal wave

$$s_k(t) = \frac{1}{m}\sin(2\pi f_k t).$$

The complete audio signal is obtained by summing all tones,

$$S(t) = \sum_k s_k(t).$$

The slight detuning creates a **chorus-like effect**, making higher multiplicities perceptually richer while preserving the pitch associated with the corresponding eigenvalue.

Consequently:

- pitch represents the eigenvalue,
- beating or wobbling represents the multiplicity of the eigenvalue.

Since high multiplicities often arise from graph symmetries, strongly symmetric graphs typically produce more pronounced beating effects.

Platonic Solids

- Tetrahedron
- Cube
- Octahedron
- Dodecahedron
- Icosahedron

Archimedean Solids

- Truncated Tetrahedron
- Truncated Cube
- Truncated Octahedron
- Truncated Dodecahedron
- Truncated Icosahedron
- Cuboctahedron
- Icosidodecahedron
- Rhombicuboctahedron
- Rhombicosidodecahedron
- Great rhombicuboctahedron
- Great rhombicosadodecahedron
- Snub cube
- Snub dodecahedron

Other graphs

- Random tree with 20 vertices
- Star graph with 10 vertices
- Path on 5 vertices
- Cycle C_5
- Complete graph K_7
- Complete Bipartite graph $K_{3,5}$

Eigenspace Sonification

The second sonification method emphasizes the **dimension of each eigenspace** rather than generating multiple detuned notes.

For an eigenvalue with multiplicity m, the harmonics of the fundamental frequency are generated:

$$f,\ 2f,\ 3f,\ \ldots,\ mf.$$

The contribution of the h^{th} harmonic is

$$s_h(t) = \frac{1}{h}\sin(2\pi h f t),$$

where the amplitude decreases proportionally to $\frac{1}{h}$.

The resulting sound is

$$S(t) = \sum_{h=1}^{m} \frac{1}{h}\sin(2\pi h f t).$$

Consequently, larger eigenspaces produce harmonically richer sounds, whereas one-dimensional eigenspaces produce nearly pure tones.

Thus:

- pitch represents the eigenvalue associated with the eigenspace,
- timbre or harmonic richness represents the dimension of the eigenspace.

Platonic Solids

- Tetrahedron
- Cube
- Octahedron

- Dodecahedron
- Icosahedron

Archimedean Solids

- Truncated Tetrahedron
- Truncated Cube
- Truncated Octahedron
- Truncated Dodecahedron
- Truncated Icosahedron
- Cuboctahedron
- Icosidodecahedron
- Rhombicuboctahedron
- Rhombicosidodecahedron
- Great rhombicuboctahedron
- Great rhombicosadodecahedron
- Snub cube
- Snub dodecahedron

Other graphs

- Random tree with 20 vertices
- Star graph with 10 vertices
- Path on 5 vertices
- Cycle C_5
- Complete graph K_7
- Complete Bipartite graph $K_{3,5}$

Applications of Graph Sonification

Graph sonification provides an auditory representation of structural graph properties and has applications in several areas of mathematics and science.

- Comparing the spectra of different graphs through sound.
- Identifying repeated eigenvalues and high-dimensional eigenspaces.
- Providing alternative representations for spectral graph analysis.
- Educational demonstrations of spectral graph theory.

- Exploratory analysis of complex networks, where auditory cues complement visual inspection.

General code used to generate audio

```
import numpy as np
from scipy.linalg import eigh
from scipy.io.wavfile import write


class SpectralSonifier:
    """
    Spectral graph sonification based on Laplacian eigenvalues
    and eigenspace multiplicities.
    """

    def __init__(self, A, name="graph", sr=44100):

        self.A = np.array(A, dtype=float)
        self.name = name
        self.sr = sr

        # Laplacian
        D = np.diag(np.sum(self.A, axis=1))
        self.L = D - self.A

        # Spectral decomposition
        self.evals, self.evecs = eigh(self.L)

    # ----------------------------------------------------
    # Utility
    # ----------------------------------------------------

    def save_wav(self, signal, filename):

        signal = signal / (np.max(np.abs(signal)) + 1e-12)

        audio = (32767 * signal).astype(np.int16)

        write(filename, self.sr, audio)

        print(f"Saved: {filename}")

    # ----------------------------------------------------
    # Group equal eigenvalues
```

```
    # ------------------------------------------------

    def spectral_blocks(self, tol=1e-6):

        blocks = []

        i = 0
        n = len(self.evals)

        while i < n:

            lam = self.evals[i]

            j = i + 1

            while j < n and abs(self.evals[j] - lam) < tol:
                j += 1

            multiplicity = j - i

            blocks.append((lam, multiplicity))

            i = j

        return blocks

    # ------------------------------------------------
    # Print spectrum
    # ------------------------------------------------

    def print_spectrum(self):

        print("\nSpectrum:\n")

        for lam, mult in self.spectral_blocks():
            print(f"  = {lam:.4f}, multiplicity = {mult}")

    # ------------------------------------------------
    # 1. Eigenvalue sonification
    # ------------------------------------------------

    def eigenvalue_sonify(self, duration=5, base_freq=220):

        t = np.linspace(0, duration, int(self.sr * duration), endpoint=False)
```

```
        signal = np.zeros_like(t)

        blocks = self.spectral_blocks()

        max_lambda = max(lam for lam, _ in blocks)

        for lam, mult in blocks:

            freq = base_freq * 2 ** (lam / (max_lambda + 1e-9))

            # multiplicity -> chord size
            for k in range(mult):

                detune = 1 + 0.01 * (k - (mult - 1) / 2)

                signal += (1 / mult) * np.sin(2 * np.pi * freq * detune * t)

        self.save_wav(signal, f"{self.name}_eigenvalues.wav")

    # ------------------------------------------------
    # 2. Eigenspace sonification
    # ------------------------------------------------

    def eigenspace_sonify(self, duration=5, base_freq=220):

        t = np.linspace(0, duration, int(self.sr * duration), endpoint=False)

        signal = np.zeros_like(t)

        blocks = self.spectral_blocks()

        max_lambda = max(lam for lam, _ in blocks)

        for lam, mult in blocks:

            freq = base_freq * 2 ** (lam / (max_lambda + 1e-9))

            # multiplicity controls richness
            for h in range(1, mult + 1):

                signal += (1 / h) * np.sin(2 * np.pi * h * freq * t)

        self.save_wav(signal, f"{self.name}_eigenspaces.wav")

    # ------------------------------------------------
```

```
    # 3. Combined sonification
    # ------------------------------------------------

    def combined_sonify(self, duration=6, base_freq=220):

        t = np.linspace(0, duration, int(self.sr * duration), endpoint=False)

        signal = np.zeros_like(t)

        blocks = self.spectral_blocks()

        max_lambda = max(lam for lam, _ in blocks)

        for lam, mult in blocks:

            freq = base_freq * 2 ** (lam / (max_lambda + 1e-9))

            # main spectral tone
            signal += np.sin(2 * np.pi * freq * t)

            # multiplicity texture
            for h in range(2, mult + 2):

                signal += (0.5 / h) * np.sin(2 * np.pi * h * freq * t)

        self.save_wav(signal, f"{self.name}_combined.wav")

    # ------------------------------------------------
    # Generate everything
    # ------------------------------------------------

    def generate_all(self):

        self.print_spectrum()

        self.eigenvalue_sonify()

        self.eigenspace_sonify()

        self.combined_sonify()

        print(f"\nFinished generating audio for {self.name}")
```

The code also generates an extra combined.wav file. The combined sonification is obtained by superposing the sounds associated with all eigenspaces. The resulting

audio provides a global auditory representation of the spectral decomposition, simultaneously reflecting the distribution of eigenvalues and the dimensions of the corresponding eigenspaces.

For simplicity purpose, the combined versions are not included here.

Back to the original page: Platonic Solids